

CEIPA Business School

Facultad de Ingenierías – Ingeniería de Sistemas

INFORME TÉCNICO

Estación 1 – Primera Entrega

Migración a Programación Orientada a Objetos

Presentado por:

Heyller De Jesús Galeano Guarín

Docente:

Simón Peláez Loaiza

Curso: Fundamentos de Software

Medellín, Junio de 2026

Tabla de contenido

1. Resumen	3
2. Introducción	3
3. Objetivos	4
4. Relación con los entregables anteriores	4
5. Identificación del problema Fintech	5
6. Archivos incluidos en el entregable	5
7. Diseño de la solución	6
8. Descripción del código fuente	7
9. Datos de prueba	8
10. Resultados obtenidos	8
11. Repositorio y acceso al docente	9
12. Estructura propuesta del archivo comprimido	10
13. Conclusiones	10
14. Referencias	11
15. Anexos	11

1. Resumen

Este informe presenta la consolidación de la Estación 1 del curso Fundamentos de Software, resultado del trabajo desarrollado en las semanas previas. El proyecto consiste en una solución para el sector Fintech que permite leer transacciones financieras desde un archivo plano, transformar cada registro en un objeto, almacenar la información en memoria y generar resultados útiles para la gestión inicial de cuentas.

La implementación final se realizó en Python mediante Programación Orientada a Objetos (POO), usando la clase Transaccion para representar cada movimiento financiero y la clase SistemaTransacciones para administrar la lectura, almacenamiento, cálculo y filtrado de datos. La entrega evidencia la migración desde la lógica estructurada hacia un diseño modular de clases y objetos que mejora la organización del código, reduce la repetición y facilita futuras ampliaciones.

2. Introducción

En el desarrollo de soluciones tecnológicas para el sector financiero, la claridad en el flujo de información es fundamental. Un sistema de gestión de cuentas debe poder registrar movimientos, leer datos de forma ordenada, calcular valores agregados y permitir consultas básicas que ayuden a interpretar el comportamiento de las transacciones. Aunque esta lógica puede construirse con programación estructurada, el crecimiento del sistema hace necesario organizar los datos y sus comportamientos en clases y objetos.

La Estación 1 propone precisamente este proceso de evolución: partir de los ejercicios previos, donde se trabajaron funciones, lectura de archivos, listas y diccionarios, y consolidar una versión orientada a objetos. La solución desarrollada lee un archivo de transacciones, crea objetos de la clase Transaccion, los administra mediante SistemaTransacciones y muestra resultados como el monto total y las transacciones filtradas por tipo.

Este informe explica la relación entre los entregables anteriores y la versión final, describe el problema Fintech abordado, presenta el diseño de la solución, documenta el código y aporta evidencias del funcionamiento y publicación en GitHub.

3. Objetivos

3.1 Objetivo general

Consolidar una solución tecnológica básica para el procesamiento de transacciones financieras, migrando la lógica de programación estructurada hacia un diseño orientado a objetos que mejore la modularidad, la comprensión y la sostenibilidad del código.

3.2 Objetivos específicos

- Definir una clase Transaccion que represente un movimiento financiero individual con sus atributos principales: cliente_id, tipo y monto.
- Diseñar una clase SistemaTransacciones encargada de leer, almacenar, mostrar, calcular y filtrar las transacciones del sistema.
- Validar la lectura del archivo plano transacciones.txt como fuente de datos principal.
- Evidenciar el funcionamiento del programa mediante capturas de pantalla y resultados obtenidos en consola.
- Publicar la solución en un repositorio privado de GitHub y compartirlo con el docente para su revisión.

4. Relación con los entregables anteriores

La Estación 1 no surge como un proyecto aislado, sino como la integración de las actividades desarrolladas en las semanas anteriores. En la Semana 1 se implementó la lógica estructurada para leer y procesar transacciones; en la Semana 2 se incorporaron los fundamentos de POO; esta entrega final combina ambos aprendizajes en un solo producto funcional. La siguiente tabla resume estos aportes:

Entregable previo	Aporte técnico	Uso en Estación 1
Semana 1 – Programación estructurada	Lectura de transacciones desde archivo, uso de listas, diccionarios, ciclos, condicionales y funciones.	Sirvió como base funcional para comprender el flujo de datos y los cálculos requeridos.
Semana 2 – Fundamentos de POO	Uso de clases, objetos, constructores, atributos y métodos para organizar la información.	Permitió migrar la solución hacia un diseño más modular y mantenible.
Quiz Semana 2 – Empresas	Lectura de datos desde archivo y creación de objetos para representar entidades del sistema.	Reforzó el patrón de lectura, validación, almacenamiento y análisis de información mediante objetos.

Tabla 1. Relación entre entregables previos y la Estación 1.

5. Identificación del problema Fintech

Una empresa del sector Fintech necesita gestionar transacciones de clientes de manera ordenada y confiable. Cada movimiento financiero contiene un identificador de cliente, un tipo de transacción y un monto. Sin una estructura de datos adecuada, pueden presentarse errores al calcular totales, filtrar operaciones o realizar modificaciones futuras al sistema.

El problema central consiste en convertir registros simples de un archivo plano en datos organizados dentro de un sistema orientado a objetos. Para atender esta necesidad, el proyecto

lee el archivo transacciones.txt, divide cada línea en sus componentes, crea un objeto Transaccion por cada registro y permite consultar la información de forma clara y estructurada. Esta lógica constituye una base fundacional para un sistema de gestión de cuentas que pueda escalar con el tiempo.

6. Archivos incluidos en el entregable

Archivo	Descripción
transaccion_poo_v1.py	Script ejecutable en Python con la versión orientada a objetos del procesador de transacciones.
transacciones.txt	Archivo plano de datos con los registros de prueba: cliente, tipo y monto.
Informe técnico	Documento académico con la argumentación del proyecto, diseño y evidencias gráficas.
Archivo .zip	Carpeta comprimida final que agrupa el código fuente, datos e informe para entrega en Canvas.

Tabla 2. Archivos que conforman la entrega de la Estación 1.

7. Diseño de la solución

La solución aplica Programación Orientada a Objetos para separar responsabilidades claramente. La clase Transaccion representa una unidad de información financiera, mientras que la clase SistemaTransacciones administra la colección de transacciones y los procesos principales. Esta separación hace que el programa sea más claro: cada objeto sabe qué información contiene y cada método tiene una responsabilidad definida.

El flujo general del sistema funciona de la siguiente manera: el archivo transacciones.txt es leído línea a línea por la clase SistemaTransacciones, que divide cada registro y lo pasa como argumentos al constructor de Transaccion. Los objetos resultantes se almacenan en una lista interna y quedan disponibles para cálculos y filtros.

7.1 Clase Transaccion

La clase Transaccion encapsula los datos de cada movimiento financiero. Sus atributos son cliente_id, tipo y monto. El constructor `__init__` recibe los valores leídos desde el archivo y convierte el monto a tipo numérico (float) para permitir operaciones matemáticas. También incluye el método `obtener_informacion()`, que retorna una representación legible de la transacción para su visualización en consola.

7.2 Clase SistemaTransacciones

La clase `SistemaTransacciones` centraliza la administración del sistema. Guarda el nombre del archivo, mantiene una lista de objetos `Transaccion` y contiene los métodos para leer datos, mostrar registros, calcular el monto total y filtrar por tipo. Esta organización evita que la lógica quede dispersa en funciones sueltas y mejora significativamente la mantenibilidad del código.

8. Descripción del código fuente

El archivo `transaccion_poo_v1.py` se organiza en tres partes: definición de clases, función principal de ejecución y llamada final al sistema. A continuación se describen los fragmentos más relevantes del código:

Definición de la clase `Transaccion`:

```
class Transaccion:
    def __init__(self, cliente_id, tipo, monto):
        self.cliente_id = cliente_id
        self.tipo = tipo
        self.monto = float(monto)

    def obtener_informacion(self):
        return f"{self.cliente_id} | {self.tipo} | ${self.monto}"
```

En este fragmento se observa la creación de la clase `Transaccion`. El uso de `self` permite guardar la información dentro del objeto, mientras que la conversión del monto a `float` garantiza que el dato pueda sumarse correctamente en operaciones posteriores.

Método `calcular_monto_total`:

```
def calcular_monto_total(self):
    total = 0
    for transaccion in self.transacciones:
        total += transaccion.monto
    return total
```

Este método recorre la lista de objetos y acumula el atributo `monto` de cada transacción. La lógica es la misma que en la versión estructurada, pero ahora se aplica sobre objetos, lo que hace el código más intuitivo y fácil de extender.

Método `filtrar_por_tipo`:

```
def filtrar_por_tipo(self, tipo_filtro):
    lista_filtrada = []
    for transaccion in self.transacciones:
        if transaccion.tipo == tipo_filtro:
            lista_filtrada.append(transaccion)
    return lista_filtrada
```

Este método aplica una estructura condicional dentro de un ciclo para seleccionar únicamente las transacciones que coinciden con el tipo solicitado. En la ejecución de prueba se filtraron las operaciones de tipo CRÉDITO.

9. Datos de prueba

El archivo transacciones.txt contiene ocho registros de prueba. Cada línea posee tres datos separados por coma: identificador del cliente, tipo de transacción y monto. Este formato permite que el programa realice una lectura simple y reproducible.

Cliente	Tipo	Monto
C001	DÉBITO	\$150.000
C002	CRÉDITO	\$250.000
C003	DÉBITO	\$80.000
C004	CRÉDITO	\$500.000
C005	DÉBITO	\$120.000
C006	DÉBITO	\$45.000
C007	CRÉDITO	\$180.000
C008	DÉBITO	\$600.000

Tabla 3. Datos del archivo transacciones.txt.

10. Resultados obtenidos

Al ejecutar el programa desde Visual Studio Code, el sistema lee correctamente las ocho transacciones, imprime cada registro en consola, calcula el monto total y muestra las transacciones del tipo CRÉDITO filtradas. El monto total obtenido fue de \$1.925.000. Las transacciones de crédito corresponden a C002, C004 y C007, con montos de \$250.000, \$500.000 y \$180.000 respectivamente.

Los resultados obtenidos son coherentes con los datos de prueba ingresados en el archivo transacciones.txt, lo que confirma que la lógica de lectura, cálculo y filtrado funciona correctamente.

[Figura 2. Evidencia de ejecución del script transaccion_poo_v1.py en Visual Studio Code.]

11. Repositorio y acceso al docente

Como parte del proceso de entrega, el proyecto fue subido a un repositorio privado de GitHub llamado Estacion1-Fundamentos-Software. En este repositorio se encuentran los archivos `transaccion_poo_v1.py` y `transacciones.txt`, los cuales permiten ejecutar y verificar la solución desarrollada.

[Figura 3. Repositorio privado en GitHub con los archivos de la Estación 1.]

Además, se compartió el repositorio con el docente mediante la opción de colaboradores de GitHub. La invitación aparece en estado pendiente, lo que evidencia que el acceso fue enviado correctamente para su revisión.

[Figura 4. Evidencia de invitación al docente SimonP8 como colaborador del repositorio.]

12. Estructura propuesta del archivo comprimido

Para la entrega en Canvas se organizó la carpeta del proyecto con la siguiente estructura, la cual permite que el evaluador encuentre rápidamente el código, los datos y el informe técnico:

```
Estacion_1_Fintech.zip
├── transaccion_poo_v1.py
├── transacciones.txt
├── Informe_Tecnico_Estacion_1.pdf
└── Informe_Tecnico_Estacion_1.docx
```

13. Conclusiones

- La solución desarrollada cumple con el objetivo de migrar una lógica estructurada hacia un diseño orientado a objetos, manteniendo la lectura del archivo, el cálculo de totales y el filtro por tipo de transacción.
- El uso de clases permite organizar mejor el código: la clase `Transaccion` representa cada movimiento financiero y la clase `SistemaTransacciones` administra las operaciones principales del sistema, separando responsabilidades de forma clara.
- La ejecución del programa demuestra que la lectura del archivo `transacciones.txt` es correcta y que los resultados obtenidos son coherentes con los datos de prueba definidos.
- La publicación en GitHub y la invitación al docente garantizan que el proyecto puede ser revisado de forma ordenada y transparente, cumpliendo con los requisitos de entrega del curso.
- La migración a POO no solo resuelve el problema planteado, sino que prepara el sistema para futuras ampliaciones como validaciones avanzadas, nuevos tipos de transacción o generación de reportes.

14. Referencias

Python Software Foundation. (2026). Python documentation. <https://docs.python.org/3/>

GitHub Docs. (2026). GitHub documentation. <https://docs.github.com/>

CEIPA. (2026). Material del curso Fundamentos de Software. Plataforma académica Canvas.

15. Anexos

Anexo A. Comando de ejecución

```
python transaccion_poo_v1.py
```

Anexo B. Contenido del archivo transacciones.txt

```
C001,DEBITO,150000  
C002,CREDITO,250000  
C003,DEBITO,80000  
C004,CREDITO,500000  
C005,DEBITO,120000  
C006,DEBITO,45000  
C007,CREDITO,180000  
C008,DEBITO,600000
```

Anexo C. Respuesta corta para sustentación

El proyecto conserva la lógica desarrollada en las semanas anteriores, pero ahora organizada con Programación Orientada a Objetos. Las transacciones ya no se manejan como datos sueltos, sino como objetos con atributos y métodos propios. Esto hace que el código sea más claro, modular y fácil de mantener a largo plazo.